

Lab 1: Identify Entities and Attributes from a Flat File

You are given a CSV export from an e-commerce app that mixes order, customer, and product details in each row. Identify the candidate entities and list their key attributes.

Sample CSV rows (columns):

order_id, order_date, customer_name, customer_email, product_id, product_name, quantity, price, shipping_address

Example:

1001,2026-06-01,Jane Doe,jane@example.com,200,Widget A,2,19.99,"123 Elm St"

Entities:

- Orders: order_id, order_date, shipping_address, amount
- Customers: customer_id, customer_name, customer_email
- Products: product_id, product_name, price
- OrderItems: order_id, product_id, price_paid, quantity

Lab 2: Classify Relationships and Keys

Using entities from Lab 1, state relationship types and propose PK/FK for each table. Include examples where orders contain multiple products.

Entity	PK	FK	Relationship
Orders	order_id	customer_id	N-1 customer, N-N products
Customers	customer_id		1-N orders
Products	product_id		N-N orderItems
OrderItems	Composite (product_id,order_id)	product_id,order_id	N-1 orders and products

Lab 3: Apply First Normal Form (1NF)

Transform the following denormalized row into 1NF rows.

One row:

order_id: 1002 customer: Bob Smith products: [{"product_id","qty"}, {"product_id","qty"}]

order_id	customer	product_id	qty
1002	Bob Smith	01	2

1002	Bob Smith	02	1
------	-----------	----	---

Lab 4: Convert Repeating Groups into Rows

Given a small JSON list of orders with product arrays, flatten into OrderItem rows using Python.

```
orders = [
    {"order_id":1003,"customer":"Ana","items":[{"product_id":301,"qty":2}]},
    {"order_id":1004,"customer":"Ben","items":[{"product_id":302,"qty":1},
                                                {"product_id":303,"qty":4}]}
]

orders = [
    {"order_id":1003,"customer":"Ana","items":[{"product_id":301,"qty":2}]},

{"order_id":1004,"customer":"Ben","items":[{"product_id":302,"qty":1},{"product_id":303,"qty":4}]}
]

flattened_orders = []
for order in orders:
    for item in order["items"]:
        new_order=order.copy()
        new_order.pop("items")
        for k,v in item.items():
            new_order[k] = v
        flattened_orders.append(new_order)

print(flattened_orders)

#OUTPUT
[{'order_id': 1003, 'customer': 'Ana', 'product_id': 301, 'qty': 2},
 {'order_id': 1004, 'customer': 'Ben', 'product_id': 302, 'qty': 1},
 {'order_id': 1004, 'customer': 'Ben', 'product_id': 303, 'qty': 4}]
```

Lab 5: Apply Second Normal Form (2NF)

You have a table **OrderItem**(**order_id**, **product_id**, **product_name**, **product_price**, **qty**). Identify partial dependencies and split tables to reach 2NF.

OrderItem(**order_id**, **product_id**, **qty**)

Products(**product_id** (PK), **product_name**, **product_price**)

OrderItem: Composite PK (**order_id**, **product_id**)

Order_id (FK)	Product_id (FK)	qty
1005	401	2
1005	402	2

Products:

Product_id (PK)	product_name	product_price
401	Gizmo	9.99
402	Doodad	5.00

Lab 6: Apply Third Normal Form (3NF)

Given **Customer**(**customer_id**, **name**, **city**, **zip**, **state_name**), identify transitive dependencies and suggest table splits to reach 3NF.

Transitive dependencies: **city** and **state_name** depends on **zip**

Customer:

customer_id(PK)	name	zip (FK)
------------------------	-------------	-----------------

ZipCodes:

zip(PK)	city	state_name
----------------	-------------	-------------------

Lab 7: Design an OLTP Normalized Schema (Orders)

Design a normalized OLTP schema for **customers**, **orders**, **order_items**, and **products**. Provide **CREATE TABLE** statements in SQL (simplified) with **PK** and **FK** constraints.

```
CREATE TABLE customers(  
    customer_id INT PRIMARY KEY,
```

```

        customer_name VARCHAR(100),
        customer_email VARCHAR(100)
    );
CREATE TABLE orders(
    order_id INT PRIMARY KEY,
    order_date DATE,
    customer_id INT,
    shipping_address VARCHAR(100),
    amount DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
CREATE TABLE order_item(
    order_id INT,
    product_id INT,
    quantity INT,
    price_paid DECIMAL(10,2),
    PRIMARY KEY (order_id, product_id),
    FOREIGN KEY (order_id ) REFERENCES orders(order_id ),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
CREATE TABLE products(
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    price DECIMAL(10,2)
);

```

Lab 8: Create a Denormalized Reporting Table

Create a denormalized daily orders reporting table that contains customer_name, order_id, order_date, total_order_value, product_list (as a single string), and shipping_city.

```

CREATE TABLE daily_order_reporting(
    customer_name VARCHAR(100),
    order_id INT PRIMARY KEY,
    order_date DATE,
    total_order_value DECIMAL(10,2),
    product_list VARCHAR(500),
    shipping_city VARCHAR(100)
);

```

```
INSERT INTO daily_order_reporting VALUES
('Vicky', 101, '2026-07-15', 299.00, 'Smartphone,Cable', 'Madrid');
```

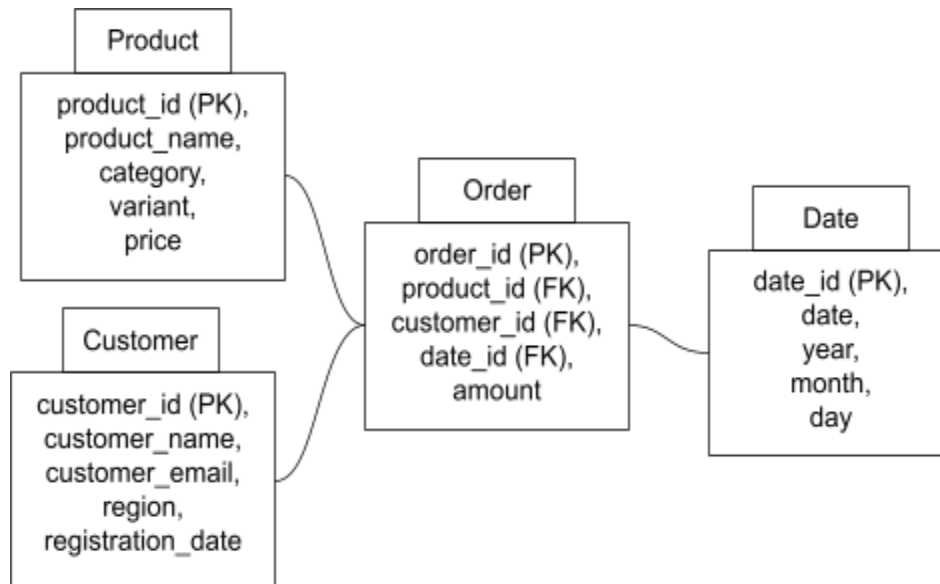
Lab 9: Design a Star Schema: Fact and Dimension Tables

Design a star schema for order analytics: FactOrders and dimensions

CustomerDim, ProductDim, DateDim. Define grain, keys, and 5 sample columns for each.

Analytics use cases: daily sales by product category and customer region.

Grain: the grain for this star schema is the order



Lab 10: Map Normalized Data to Star Schema (ETL sketch)

Sketch SQL/ pseudocode to load fact table from normalized OLTP tables: orders, order_items, products, customers. Include joins and aggregations for daily fact (one row per order_item with sales_amount).

```
INSERT INTO daily_order_item_reporting(
    order_id,
    product_id,
    order_date,
    order_item,
    total_order_value
)
SELECT
    o.order_id,
    P.product_id,
```

```

        o.order_date,
        p.product_name as order_item,
        oi.quantity*oi.price_paid AS total_order_amount
FROM orderItem oi
LEFT JOIN products p ON oi.product_id = p.product_id
LEFT JOIN orders o on oi.order_id = o.order_id

```

Lab 11: Model Slowly Changing Dimension (Type 2)

Design a CustomerDim that captures address history using Type 2 SCD. Show table columns and example rows demonstrating a customer moving city.

Surrogate_id	Customer_id	customer_name	address	start_date	end_date	in_use
1	101	Alex	Main Street 1	2025-04-01	2026-05-01	False
2	101	Alex	Captain Street	2026-05-01		True

Lab 12: Design for Query Performance: Denormalization Trade-offs

Given slow join-heavy reports, propose three denormalization changes to speed up queries and list the trade-offs for each.

Scenario / Input

Reports frequently join orders -> customers -> addresses -> product -> category.

1. Union last addresses to the customers table: this avoids the join between customers and addresses
2. Union category to the product table: this avoids the join between product and category
3. Create a table with some aggregations. For example: Total customer per city. Avg price per category.